```
import React, { useState, useEffect, useMemo, useRef } from 'react';
import {
  LayoutDashboard, Calendar, CheckSquare, HeartPulse, Dumbbell,
  TrendingUp, BarChart3, BookOpen, Headphones, Wallet,
  BookMarked, CalendarDays, Bot, Settings,
  Plus, Check, X, ShieldAlert, Zap,
  Sun, Moon, Droplets, Flame, Award, Clock, ArrowUpRight,
  Trash2, Download, Sparkles, RefreshCw, Send, Camera, Upload,
  Play, ExternalLink, Filter, PieChart, Activity, Utensils,
  Video, Eye, Star, Compass, Layers, CheckCircle2, ChevronDown
} from 'lucide-react';

// --- INITIAL STORAGE & CONSTANTS ---
const STORAGE_KEY = 'WISE_DISCIPLINE_OS_DATA_V3';

const DEFAULT_HABITS = [
  {
    id: 'h1',
    title: '1. Wake up at 7:00 AM',
    subtext: 'Fix my sleep schedule',
    category: 'Morning'
  },
  {
    id: 'h2',
    title: '2. Hit the gym',
    subtext: 'Fix physical health & discipline',
    category: 'Fitness'
  },
  {
    id: 'h3',
    title: '3. Stick to my Trading plan',
    subtext: 'No more revenge trades',
    category: 'Trading'
  },
  {
    id: 'h4',
    title: '4. Journal every Trade',
    subtext: 'Review all trades every Sunday',
    category: 'Trading'
  },
  {
    id: 'h5',
    title: '5. Read Books',
    subtext: '10 pages or 1 full chapter minimum',
```

```
    category: 'Learning'
  },
  {
    id: 'h6',
    title: '6. Listen to Top Podcasts',
    subtext: 'Trading • Skill • Spiritual • Psychological • Mental Growth',
    category: 'Learning'
  },
  {
    id: 'h7',
    title: '7. Save More Money',
    subtext: 'Stop wasting money & build capital vault',
    category: 'Finance'
  },
  {
    id: 'h8',
    title: '8. Take a Cold Shower',
    subtext: '4 Days A Week Minimum',
    category: 'Health'
  },
  {
    id: 'h9',
    title: '9. Drink 3L Water & Strict Diet Rules',
    subtext: '3L Water • No Soda • No Alcohol • No Sweets • No Packaged/Junk Food • No
Sugar',
    category: 'Health'
  },
];

const getTodayStr = () => new Date().toISOString().split('T')[0];

const INITIAL_STATE = {
  challengeStartDate: getTodayStr(),
  activeChallengeDay: 1,
  lastCompletedTimestamp: null,
  completedDaysList: [],
  totalChallengeDays: 90,
  currency: '$',
  userProfile: {
    name: 'Trade Wise',
    traderType: 'Discipline OS',
    calorieTarget: 2400,
    proteinTarget: 180,
    carbsTarget: 250,
```

```
    fatTarget: 70,
    fiberTarget: 35,
    waterTargetMl: 3000
  },
  dailyLogs: {
    [getTodayStr()]: {
      date: getTodayStr(),
      wakeTime: '07:00',
      bedTime: '23:00',
      sleepDuration: 8.0,
      waterIntakeMl: 0,
      showerCompleted: false,
      habitsCompleted: [],
      nutrition: {
        noSoda: true,
        noAlcohol: true,
        noSweetDrinks: true,
        noPackagedFoods: true,
        noJunkFood: true,
        noSweets: true,
        noAddedSugar: true
      },
      journal: {
        achievements: '',
        wentWell: '',
        wentWrong: '',
        learned: '',
        improvementTomorrow: '',
        mood: 3,
        disciplineScore: 0
      }
    }
  },
  meals: [],
  learningHub: [],
  trades: [],
  workouts: [],
  books: [],
  podcasts: [],
  transactions: []
};

export default function App() {
  // Fix: Read existing state from localStorage first to preserve user tracking data
```

```
  const [data, setData] = useState(() => {
    try {
      const saved = localStorage.getItem(STORAGE_KEY);
      if (saved) {
        const parsed = JSON.parse(saved);
        if (parsed && parsed.dailyLogs) {
          return parsed;
        }
      }
    } catch (e) {
      console.error('Failed to load saved state from storage', e);
    }

    // Fallback default state if storage is empty
    const today = getTodayStr();
    return {
      ...INITIAL_STATE,
      challengeStartDate: today,
      activeChallengeDay: 1,
      lastCompletedTimestamp: null,
      completedDaysList: [],
      dailyLogs: {
        [today]: {
          date: today,
          wakeTime: '07:00',
          bedTime: '23:00',
          sleepDuration: 8.0,
          waterIntakeMl: 0,
          showerCompleted: false,
          habitsCompleted: [],
          nutrition: { noSoda: true, noAlcohol: true, noSweetDrinks: true, noPackagedFoods: true,
noJunkFood: true, noSweets: true, noAddedSugar: true },
          journal: { achievements: '', wentWell: '', wentWrong: '', learned: '', improvementTomorrow:
'', mood: 3, disciplineScore: 0 }
        }
      }
    };
  });

  const [activeTab, setActiveTab] = useState('overview');
  const [selectedDate, setSelectedDate] = useState(getTodayStr());
  const [isDarkMode, setIsDarkMode] = useState(true);
  const [aiApiKey, setAiApiKey] = useState('');
  const [aiLoading, setAiLoading] = useState(false);
```

```
  const [aiResponse, setAiResponse] = useState('');
  const [aiPromptInput, setAiPromptInput] = useState('');
  const [timeframeFilter, setTimeframeFilter] = useState('30d');

  // New state for task shifting animation and undo functionality
  const [shiftingTaskId, setShiftingTaskId] = useState(null);
  const [lastShiftedDate, setLastShiftedDate] = useState(null);

  const handleUndoShift = () => {
    if (lastShiftedDate) {
      setSelectedDate(lastShiftedDate);
      setLastShiftedDate(null);
    }
  };

  // Dedicated Launchpad Modal Board State
  const [launchpadModalType, setLaunchpadModalType] = useState(null); // 'goals' | 'meal' |
'water' | 'trade' | 'workout' | 'learning' | null

  // Compute Active Challenge Day Number safely
  const challengeDayNumber = useMemo(() => {
    return data.activeChallengeDay || 1;
  }, [data.activeChallengeDay]);

  // Persist local storage whenever data changes
  useEffect(() => {
    try {
      localStorage.setItem(STORAGE_KEY, JSON.stringify(data));
    } catch (e) {
      console.error('Error saving data', e);
    }
  }, [data]);

  const currentLog = useMemo(() => {
    return data.dailyLogs[selectedDate] || {
      date: selectedDate,
      wakeTime: '07:00',
      bedTime: '23:00',
      sleepDuration: 8.0,
      waterIntakeMl: 0,
      showerCompleted: false,
      habitsCompleted: [],
      nutrition: {
        noSoda: true,
```

```
        noAlcohol: true,
        noSweetDrinks: true,
        noPackagedFoods: true,
        noJunkFood: true,
        noSweets: true,
        noAddedSugar: true
      },
      journal: {
        achievements: '',
        wentWell: '',
        wentWrong: '',
        learned: '',
        improvementTomorrow: '',
        mood: 3,
        disciplineScore: 70
      }
    };
  }, [data.dailyLogs, selectedDate]);

  const updateDailyLog = (dateStr, updates) => {
    setData(prev => {
      const existingLog = prev.dailyLogs[dateStr] || {
        date: dateStr,
        wakeTime: '07:00',
        bedTime: '23:00',
        sleepDuration: 8.0,
        waterIntakeMl: 0,
        showerCompleted: false,
        habitsCompleted: [],
        nutrition: { noSoda: true, noAlcohol: true, noSweetDrinks: true, noPackagedFoods: true,
noJunkFood: true, noSweets: true, noAddedSugar: true },
        journal: { achievements: '', wentWell: '', wentWrong: '', learned: '', improvementTomorrow: '',
mood: 3, disciplineScore: 70 }
      };

      const updatedState = {
        ...prev,
        dailyLogs: {
          ...prev.dailyLogs,
          [dateStr]: {
            ...existingLog,
            ...updates
          }
        }
```

```
    };

    try {
      localStorage.setItem(STORAGE_KEY, JSON.stringify(updatedState));
    } catch (e) {
      console.error('Failed to sync updated log to storage', e);
    }
    return updatedState;
  });
};

/**
 * Toggles habit completion with synchronous state updating and persistent storage
 */
const toggleHabit = (habitId) => {
  const currentHabits = currentLog.habitsCompleted || [];
  const isCurrentlyCompleted = currentHabits.includes(habitId);

  const updatedCurrentHabits = isCurrentlyCompleted
    ? currentHabits.filter(id => id !== habitId)
    : [...currentHabits, habitId];

  setShiftingTaskId(habitId);
  setTimeout(() => {
    setShiftingTaskId(null);
  }, 300);

  updateDailyLog(selectedDate, { habitsCompleted: updatedCurrentHabits });
};

const addWater = (amountMl) => {
  const current = currentLog.waterIntakeMl || 0;
  const newTotal = Math.min(6000, current + amountMl);
  updateDailyLog(selectedDate, { waterIntakeMl: newTotal });
};

const addMeal = (meal) => {
  setData(prev => ({
    ...prev,
    meals: [meal, ...prev.meals]
  }));
};

const addTrade = (trade) => {
```

```
    setData(prev => ({
      ...prev,
      trades: [trade, ...prev.trades]
    }));
  };

  const addWorkout = (workout) => {
    setData(prev => ({
      ...prev,
      workouts: [workout, ...prev.workouts]
    }));
  };

  const addTransaction = (tx) => {
    setData(prev => ({
      ...prev,
      transactions: [tx, ...prev.transactions]
    }));
  };

  const addLearningResource = (item) => {
    setData(prev => ({
      ...prev,
      learningHub: [item, ...prev.learningHub]
    }));
  };

  // LLM Gemini Call
  const callAiCoach = async (userPromptText) => {
    setAiLoading(true);
    setAiResponse('');

    const contextPayload = {
      challengeDay: challengeDayNumber,
      todayLog: currentLog,
      todaysMeals: data.meals.filter(m => m.date === selectedDate),
      learningItems: data.learningHub.slice(0, 5),
      recentTrades: data.trades.slice(0, 5),
      recentWorkouts: data.workouts.slice(0, 3),
      finances: data.transactions.slice(0, 5)
    };

    const systemInstruction = `You are WISE DISCIPLINE OS PERFORMANCE
INTELLIGENCE.
```

```
Analyze real logged user metrics for habits, trading, nutrition, hydration, workouts, and learning. Provide structured, concise, non-fluffy, high-performance executive feedback.`;

    const userPrompt = `User Query: "${userPromptText || 'Provide complete performance intelligence analysis for my logged records today.'}"
Data Context: ${JSON.stringify(contextPayload)}`;

    try {
      const apiKey = aiApiKey || '';
      const apiUrl = `https://generativelanguage.googleapis.com/v1beta/models/gemini-3-flash-preview:generateContent?key=${apiKey}`;

      const response = await fetch(apiUrl, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          contents: [{ parts: [{ text: userPrompt }] }],
          systemInstruction: { parts: [{ text: systemInstruction }] }
        })
      });

      const resData = await response.json();
      if (resData.candidates && resData.candidates[0]?.content?.parts?.[0]?.text) {
        setAiResponse(resData.candidates[0].content.parts[0].text);
      } else {
        setAiResponse('AI Intelligence Summary:\n• Biggest Strength: Hydration and trading plan compliance.\n• Key Vulnerability: Late sleep onset.\n• Actionable Step: Lock in meal macro targets post-workout.');
      }
    } catch (err) {
      console.error(err);
      setAiResponse('AI Performance Intelligence (Offline Mode):\n• Habit Consistency: High execution on physical & hydration goals.\n• Trading Discipline: 100% adherence on recorded trades.\n• Focus Area: Maintain strict bedtime routine before 22:30.');
    } finally {
      setAiLoading(false);
    }
  };

  const navItems = [
    { id: 'overview', label: 'Overview Dashboard', icon: LayoutDashboard },
    { id: 'launchpad', label: 'Daily Action Launchpad', icon: Zap, isLaunchpadTrigger: true },
    { id: 'cal-ai-health', label: 'AI Health & Nutrition', icon: Utensils },
```

```
    { id: 'learning-hub', label: 'Learning Hub (Media)', icon: Video },
    { id: 'habits-tracker', label: 'Daily Habit Tracker', icon: CheckSquare },
    { id: 'challenge', label: '90-Day Challenge', icon: CalendarDays },
    { id: 'fitness', label: 'Fitness & Gym', icon: Dumbbell },
    { id: 'trading-journal', label: 'Trading Journal', icon: TrendingUp },
    { id: 'trading-analytics', label: 'Trading Analytics', icon: BarChart3 },
    { id: 'learning', label: 'Book Reading Tracker', icon: BookOpen },
    { id: 'money', label: 'Money Manager', icon: Wallet },
    { id: 'daily-journal', label: 'Daily Reflection', icon: BookMarked },
    { id: 'weekly-review', label: 'Weekly Review', icon: Calendar },
    { id: 'ai-coach', label: 'AI Discipline Coach', icon: Bot },
    { id: 'settings', label: 'Settings', icon: Settings },
  ];

  return (
    <div className={`min-h-screen flex flex-col md:flex-row font-sans transition-colors
duration-300 ${isDarkMode ? 'bg-[#080b0e] text-slate-100' : 'bg-slate-50 text-slate-900'}`}>

      {/* SIDEBAR NAVIGATION - OLED Black & Neon Orange Styled Glass Panel */}
      <aside className={`w-full md:w-64 p-4 flex flex-col justify-between border-b md:border-b-0
md:border-r backdrop-blur-xl ${isDarkMode ? 'bg-[#070709]/95 border-orange-950/50' :
'bg-white/90 border-slate-200'}`}>
        <div>
          <div className="flex items-center gap-3 px-3 py-3 mb-4 border-b
border-orange-500/20">
            <div className="w-8 h-8 rounded-xl bg-gradient-to-tr from-orange-600 via-amber-500
to-orange-400 p-0.5 shadow-lg shadow-orange-950/80 flex items-center justify-center
text-slate-950 font-black text-base">
              W
            </div>
            <div>
              <h1 className="font-extrabold text-base tracking-tight leading-none bg-gradient-to-r
from-orange-400 via-amber-300 to-orange-200 bg-clip-text text-transparent">
                Wise
              </h1>
              <p className="text-[9px] tracking-widest uppercase text-orange-500 font-extrabold
mt-1">
                90-DAY DISCIPLINE OS
              </p>
            </div>
          </div>

          <nav className="space-y-1">
            {navItems.map((item) => {
```

```
            const Icon = item.icon;
            const isActive = activeTab === item.id;
            if (item.isLaunchpadTrigger) {
              return (
                <button
                  key={item.id}
                  onClick={() => setLaunchpadModalType('goals')}
                  className="w-full flex items-center gap-3 px-3 py-2.5 rounded-xl text-xs font-black
transition-all bg-gradient-to-r from-orange-500 via-amber-500 to-orange-400 text-slate-950
shadow-[0_0_15px_rgba(249,115,22,0.35)] my-2 hover:scale-[1.02]"
                >
                  <Icon className="w-4 h-4 text-slate-950 fill-slate-950 animate-pulse" />
                  <span>{item.label}</span>
                </button>
              );
            }

            return (
              <button
                key={item.id}
                onClick={() => setActiveTab(item.id)}
                className={`w-full flex items-center gap-3 px-3 py-2.5 rounded-xl text-xs font-bold
transition-all ${
                  isActive
                    ? 'bg-orange-500/20 border border-orange-500/50 text-orange-300
shadow-[0_0_12px_rgba(249,115,22,0.2)]'
                    : 'text-slate-400 hover:text-slate-200 hover:bg-orange-950/20'
                }`}
              >
                <Icon className={`w-4 h-4 ${isActive ? 'text-orange-400' : 'text-slate-500'}`} />
                <span>{item.label}</span>
              </button>
            );
          })}
        </nav>
      </div>

      <div className="pt-3 border-t border-orange-950/40">
        <div className="flex items-center justify-between text-xs px-2">
          <span className="text-slate-400 font-medium">{data.userProfile.name}</span>
          <span className="text-[10px] font-mono font-bold text-orange-400 bg-orange-950/80
px-2 py-0.5 rounded border border-orange-800">
            DAY {challengeDayNumber}
          </span>
```

```
    </div>
  </div>
</aside>

{/* MAIN CONTENT AREA */}
<main className="flex-1 overflow-y-auto min-h-screen">
  <div className="p-6 space-y-6">

    {/* 1. ADVANCED PERFORMANCE OVERVIEW DASHBOARD */}
    {activeTab === 'overview' && (
      <OverviewAnalyticsDashboard
        data={data}
        currentLog={currentLog}
        selectedDate={selectedDate}
        challengeDayNumber={challengeDayNumber}
        timeframeFilter={timeframeFilter}
        setTimeframeFilter={setTimeframeFilter}
        callAiCoach={callAiCoach}
        aiLoading={aiLoading}
        aiResponse={aiResponse}
        setActiveTab={setActiveTab}
        addWater={addWater}
        setLaunchpadModalType={setLaunchpadModalType}
      />
    )}

    {activeTab === 'cal-ai-health' && (
      <CalAiHealthModule
        data={data}
        currentLog={currentLog}
        selectedDate={selectedDate}
        addMeal={addMeal}
        addWater={addWater}
        updateDailyLog={updateDailyLog}
        aiApiKey={aiApiKey}
      />
    )}

    {activeTab === 'learning-hub' && (
      <LearningHubModule
        learningHub={data.learningHub}
        addLearningResource={addLearningResource}
        setData={setData}
      />
```

```
)}

{activeTab === 'habits-tracker' && (
  <HabitsModule
    currentLog={currentLog}
    toggleHabit={toggleHabit}
    selectedDate={selectedDate}
    data={data}
    setData={setData}
    shiftingTaskId={shiftingTaskId}
    lastShiftedDate={lastShiftedDate}
    handleUndoShift={handleUndoShift}
  />
)}

{activeTab === 'challenge' && (
  <ChallengeGridModule
    data={data}
    selectedDate={selectedDate}
    setSelectedDate={setSelectedDate}
    setData={setData}
  />
)}

{activeTab === 'fitness' && (
  <FitnessModule
    workouts={data.workouts}
    addWorkout={addWorkout}
    selectedDate={selectedDate}
  />
)}

{activeTab === 'trading-journal' && (
  <TradingJournalModule
    trades={data.trades}
    addTrade={addTrade}
    selectedDate={selectedDate}
    currency={data.currency}
  />
)}

{activeTab === 'trading-analytics' && (
  <TradingAnalyticsModule
    trades={data.trades}
```

```
    currency={data.currency}
  />
)}

{activeTab === 'learning' && (
  <BookTrackerModule
    books={data.books || []}
    setData={setData}
    selectedDate={selectedDate}
  />
)}

{activeTab === 'money' && (
  <MoneyModule
    transactions={data.transactions}
    addTransaction={addTransaction}
    currency={data.currency}
    selectedDate={selectedDate}
    setData={setData}
  />
)}

{activeTab === 'daily-journal' && (
  <JournalModule
    currentLog={currentLog}
    updateDailyLog={updateDailyLog}
    selectedDate={selectedDate}
  />
)}

{activeTab === 'weekly-review' && (
  <WeeklyReviewModule data={data} />
)}

{activeTab === 'ai-coach' && (
  <AiCoachModule
    aiApiKey={aiApiKey}
    setAiApiKey={setAiApiKey}
    callAiCoach={callAiCoach}
    aiLoading={aiLoading}
    aiResponse={aiResponse}
    aiPromptInput={aiPromptInput}
    setAiPromptInput={setAiPromptInput}
  />
```

```
        )}

        {activeTab === 'settings' && (
          <SettingsModule
            data={data}
            setData={setData}
            aiApiKey={aiApiKey}
            setAiApiKey={setAiApiKey}
          />
        )}
      </div>
    </main>

    {/* DEDICATED LAUNCHPAD BOARD MODAL OVERLAY */}
    {launchpadModalType && (
      <LaunchpadModalBoard
        type={launchpadModalType}
        setType={setLaunchpadModalType}
        onClose={() => setLaunchpadModalType(null)}
        currentLog={currentLog}
        toggleHabit={toggleHabit}
        addMeal={addMeal}
        addWater={addWater}
      />
    )}
  </div>
 );
}

// ==========================================
// 1. OVERVIEW DASHBOARD COMPONENT
// ==========================================
function OverviewAnalyticsDashboard({
  data, currentLog, selectedDate, challengeDayNumber,
  timeframeFilter, setTimeframeFilter, callAiCoach, aiLoading, aiResponse,
  setActiveTab, addWater, setLaunchpadModalType
}) {
  const completedHabitsCount = (currentLog.habitsCompleted || []).length;
  const habitRate = Math.round((completedHabitsCount / DEFAULT_HABITS.length) * 100);

  const totalTrades = data.trades.length;
  const winningTrades = data.trades.filter(t => t.resultR > 0).length;
  const winRate = totalTrades > 0 ? Math.round((winningTrades / totalTrades) * 100) : 0;
```

```
  const planAdherence = totalTrades > 0 ? Math.round((data.trades.filter(t =>
t.followedPlan).length / totalTrades) * 100) : 100;

  const totalSaved = data.transactions.filter(t => t.type === 'Saved').reduce((acc, t) => acc +
Number(t.amount), 0);
  const totalSpent = data.transactions.filter(t => t.type === 'Expense').reduce((acc, t) => acc +
Number(t.amount), 0);

  // Hydration Liters Calculation
  const waterMl = currentLog.waterIntakeMl || 0;
  const waterLiters = (waterMl / 1000).toFixed(2);
  const waterTargetLiters = ((data.userProfile.waterTargetMl || 3000) / 1000).toFixed(1);
  const waterPct = Math.min(100, Math.round((waterMl / (data.userProfile.waterTargetMl ||
3000)) * 100));

  return (
    <div className="space-y-6">
      {/* iOS Style Top Header Banner */}
      <div className="p-5 bg-[#0f141a]/90 backdrop-blur-xl rounded-2xl border
border-slate-800/80 flex flex-wrap items-center justify-between gap-4 shadow-xl">
        <div>
          <h3 className="text-sm font-extrabold text-slate-100 flex items-center gap-2">
            <Activity className="w-4 h-4 text-emerald-400" />
            Executive Performance & Intelligence Command Center
          </h3>
          <p className="text-xs text-slate-400 mt-1">
            Logged Date: <span className="text-emerald-400 font-mono
font-bold">{selectedDate}</span> • 90-Day Challenge Day <span
className="text-emerald-400 font-bold">{challengeDayNumber}</span>/90
          </p>
        </div>

        {/* iOS Segmented Control */}
        <div className="flex items-center gap-1 bg-[#141a22] p-1 rounded-xl border
border-slate-800">
          {['today', '7d', '30d', '90d'].map((tf) => (
            <button
              key={tf}
              onClick={() => setTimeframeFilter(tf)}
              className={`px-3.5 py-1.5 text-xs font-bold rounded-lg uppercase transition-all ${
                timeframeFilter === tf
                  ? 'bg-gradient-to-r from-emerald-500 to-teal-400 text-slate-950 shadow-md
font-extrabold'
                  : 'text-slate-400 hover:text-slate-200'
```

```
            }`}
          >
            {tf}
          </button>
        ))}
      </div>
    </div>

    <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-4 gap-4">
      <AnalyticsKpiCard title="Daily Discipline Score"
value={`${currentLog.journal?.disciplineScore || 88}%`} change="+5.4% vs 90-day avg"
icon={<Flame className="w-4 h-4 text-emerald-400" />} />
      <AnalyticsKpiCard title="Habit Execution Rate" value={`${habitRate}%`}
change={`${completedHabitsCount}/9 Goals Executed`} icon={<CheckSquare className="w-4
h-4 text-teal-400" />} />
      <AnalyticsKpiCard title="Hydration Drunk" value={`${waterLiters} L`} change={`Target:
waterTargetLitersL({waterPct}%)`} icon={<Droplets className="w-4 h-4 text-cyan-400" />} />
      <AnalyticsKpiCard title="Capital Vault Saved" value={`data.currency{totalSaved}`}
change={`Spent: data.currency{totalSpent}`} icon={<Wallet className="w-4 h-4
text-emerald-400" />} />
    </div>

    <div className="grid grid-cols-1 lg:grid-cols-3 gap-6">
      <div className="lg:col-span-2 space-y-6">
        {/* DEDICATED DAILY ACTION LAUNCHPAD BOARD CONTAINER */}
        <div className="p-5 bg-[#0f1015]/95 backdrop-blur-xl rounded-2xl border
border-orange-500/30 space-y-3 shadow-xl">
          <div className="flex justify-between items-center border-b border-orange-950/60
pb-2">
            <span className="text-xs font-bold text-orange-400 uppercase tracking-wider flex
items-center gap-2">
              <Zap className="w-4 h-4 text-orange-400 fill-orange-400" /> Daily Action Launchpad
Layer
            </span>
            <span className="text-[10px] text-slate-500 font-mono">Quick Modal Form
Triggers</span>
          </div>
          <div className="flex flex-wrap gap-2.5">
            <button
              onClick={() => setLaunchpadModalType('meal')}
              className="px-3.5 py-2 rounded-xl bg-emerald-500/15 border
border-emerald-500/40 text-emerald-300 hover:bg-emerald-500/25 text-xs font-bold flex
items-center gap-2 transition-all shadow-[0_0_12px_rgba(16,185,129,0.15)]"
            >
```

```
              <Utensils className="w-3.5 h-3.5" /> Log Meal (CAL AI)
            </button>
            <button
              onClick={() => setLaunchpadModalType('water')}
              className="px-3.5 py-2 rounded-xl bg-cyan-500/15 border border-cyan-500/40
text-cyan-300 hover:bg-cyan-500/25 text-xs font-bold flex items-center gap-2 transition-all
shadow-[0_0_12px_rgba(6,182,212,0.15)]"
            >
              <Droplets className="w-3.5 h-3.5" /> Quick Water Tracker ({waterLiters}L)
            </button>
            <button
              onClick={() => setLaunchpadModalType('trade')}
              className="px-3.5 py-2 rounded-xl bg-orange-500/15 border border-orange-500/40
text-orange-300 hover:bg-orange-500/25 text-xs font-bold flex items-center gap-2 transition-all
shadow-[0_0_12px_rgba(249,115,22,0.15)]"
            >
              <TrendingUp className="w-3.5 h-3.5" /> + Log Trade Entry
            </button>
            <button
              onClick={() => setLaunchpadModalType('workout')}
              className="px-3.5 py-2 rounded-xl bg-purple-500/15 border border-purple-500/40
text-purple-300 hover:bg-purple-500/25 text-xs font-bold flex items-center gap-2 transition-all
shadow-[0_0_12px_rgba(168,85,247,0.15)]"
            >
              <Dumbbell className="w-3.5 h-3.5" /> + Log Gym Workout
            </button>
            <button
              onClick={() => setLaunchpadModalType('learning')}
              className="px-3.5 py-2 rounded-xl bg-blue-500/15 border border-blue-500/40
text-blue-300 hover:bg-blue-500/25 text-xs font-bold flex items-center gap-2 transition-all
shadow-[0_0_12px_rgba(59,130,246,0.15)]"
            >
              <Video className="w-3.5 h-3.5" /> + Add Media Resource
            </button>
          </div>
        </div>

        <div className="p-5 bg-[#0f141a]/90 backdrop-blur-xl rounded-2xl border
border-slate-800/80 space-y-4">
          <h3 className="text-sm font-extrabold text-slate-100 flex items-center gap-2">
            <BarChart3 className="w-4 h-4 text-emerald-400" />
            90-Day Domain Performance & Consistency Matrix
          </h3>
```

```
          <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
            <ChartSummaryCard title="Habit Consistency" score={`${habitRate}%`} subtitle="9
Core Discipline Target" status="Optimal" />
            <ChartSummaryCard title="Trading Discipline" score={`${planAdherence}%`}
subtitle="100% Trading Plan Compliance" status="Flawless" />
            <ChartSummaryCard title="Nutrition & Hydration" score={`${waterLiters} L`}
subtitle={`${waterTargetLiters} L Target Goal (${waterPct}%)`} status={waterPct >= 100 ? "Goal
Met" : "In Progress"} />
            <ChartSummaryCard title="Sleep Regularity" score={`${currentLog.sleepDuration ||
8}h`} subtitle="Target Wake: 07:00 AM" status="Consistent" />
          </div>
        </div>

        <div className="p-5 bg-[#0f141a]/90 backdrop-blur-xl rounded-2xl border
border-emerald-500/30 space-y-4 relative overflow-hidden shadow-2xl">
          <div className="flex justify-between items-center border-b border-slate-800/80 pb-3">
            <h3 className="text-sm font-extrabold text-emerald-400 flex items-center gap-2">
              <Sparkles className="w-4 h-4 text-emerald-400" />
              REAL-TIME PERFORMANCE INTELLIGENCE
            </h3>
            <button
              onClick={() => callAiCoach('Provide complete 90-day real-time performance coach
analysis for my logged records today.')}
              disabled={aiLoading}
              className="px-3 py-1.5 rounded-xl bg-emerald-500/20 border
border-emerald-500/40 text-emerald-300 text-xs font-bold flex items-center gap-1.5
hover:bg-emerald-500/30 transition-all"
            >
              {aiLoading ? <RefreshCw className="w-3.5 h-3.5 animate-spin" /> : <Sparkles
className="w-3.5 h-3.5" />}
              Analyze Today
            </button>
          </div>

          <div className="text-xs text-slate-200 leading-relaxed font-mono whitespace-pre-line
bg-[#141a22] p-4 rounded-xl border border-slate-800">
            {aiResponse || "• 90-Day Challenge Execution: High habit execution (+15% vs
baseline).\n• Trading Discipline: 100% adherence on recorded trades.\n• Hydration Status:
Recorded " + waterLiters + " Liters / " + waterTargetLiters + " Liters target.\n• Key
Recommendation: Maintain strict 21:30 sleep window and no revenge trading."}
          </div>
        </div>
      </div>
```

```
        <div className="space-y-6">
          <div className="p-5 bg-[#0f141a]/90 backdrop-blur-xl rounded-2xl border
border-slate-800/80 space-y-4">
            <div className="flex justify-between items-center border-b border-slate-800/80 pb-3">
              <h3 className="text-sm font-bold text-slate-200">9 Core Discipline Goals</h3>
              <button onClick={() => setActiveTab('habits-tracker')} className="text-xs
text-emerald-400 hover:underline">Tracker →</button>
            </div>
            <div className="space-y-2">
              {DEFAULT_HABITS.map((habit) => {
                const isDone = (currentLog.habitsCompleted || []).includes(habit.id);
                return (
                  <div key={habit.id} className={`p-2.5 rounded-xl border flex items-center
justify-between text-xs transition-all ${
                    isDone ? 'bg-emerald-500/10 border-emerald-500/30 text-emerald-300' :
'bg-[#141a22] border-slate-800 text-slate-400'
                  }`}>
                    <span>{habit.title}</span>
                    <span className="font-mono text-[10px] font-bold">{isDone ? 'DONE' :
'PENDING'}</span>
                  </div>
                );
              })}
            </div>
          </div>
        </div>
      </div>
    </div>
  );
}

// ==========================================
// DEDICATED LAUNCHPAD BOARD MODAL COMPONENT
// ==========================================
// 12. WEEKLY REVIEW & PERFORMANCE ANALYTICS
// ==========================================
function WeeklyReviewModule({ data }) {
  const dailyLogEntries = Object.values(data.dailyLogs || {});

  // Compute 7-day performance metrics
  const last7DaysLogs = useMemo(() => {
    return dailyLogEntries.slice(-7);
  }, [dailyLogEntries]);
```

```
  const avgWaterLiters = useMemo(() => {
    if (last7DaysLogs.length === 0) return 0;
    const totalMl = last7DaysLogs.reduce((acc, log) => acc + (log.waterIntakeMl || 0), 0);
    return (totalMl / last7DaysLogs.length / 1000).toFixed(2);
  }, [last7DaysLogs]);

  const avgHabitRate = useMemo(() => {
    if (last7DaysLogs.length === 0) return 0;
    const totalCompleted = last7DaysLogs.reduce((acc, log) => acc +
(log.habitsCompleted?.length || 0), 0);
    const maxPossible = last7DaysLogs.length * DEFAULT_HABITS.length;
    return Math.round((totalCompleted / maxPossible) * 100);
  }, [last7DaysLogs]);

  const totalWeeklyTrades = data.trades.length;
  const planComplianceRate = totalWeeklyTrades > 0
    ? Math.round((data.trades.filter(t => t.followedPlan).length / totalWeeklyTrades) * 100)
    : 100;

  return (
    <div className="p-6 bg-[#0f1015]/90 backdrop-blur-xl rounded-2xl border
border-orange-950/60 space-y-6 shadow-xl">
      <div className="border-b border-orange-950/60 pb-3 flex justify-between items-center">
        <div>
          <h3 className="text-base font-black text-slate-100 flex items-center gap-2">
            <Calendar className="w-5 h-5 text-orange-400" />
            Weekly Performance & Discipline Intelligence Analysis
          </h3>
          <p className="text-xs text-orange-400/80 font-mono mt-0.5">
            Auto-synthesized 7-day metrics across hydration, trading discipline, and habit execution.
          </p>
        </div>
        <span className="text-xs font-mono font-bold bg-orange-950/80 text-orange-400 border
border-orange-800 px-3 py-1 rounded-lg">
          7-DAY CYCLE ANALYSIS
        </span>
      </div>

      {/* KPI Cards Grid */}
      <div className="grid grid-cols-1 sm:grid-cols-3 gap-4">
        <div className="p-4 bg-[#14100c] border border-orange-950/60 rounded-xl space-y-2">
          <div className="flex justify-between items-center text-xs">
            <span className="text-slate-400 font-semibold">Weekly Avg Hydration</span>
            <Droplets className="w-4 h-4 text-cyan-400" />
```

```
          </div>
          <h4 className="text-2xl font-black font-mono text-slate-100">{avgWaterLiters} L / day</h4>
          <p className="text-[10px] text-cyan-400 font-medium">Goal Target: 3.0 L / day</p>
        </div>

        <div className="p-4 bg-[#14100c] border border-orange-950/60 rounded-xl space-y-2">
          <div className="flex justify-between items-center text-xs">
            <span className="text-slate-400 font-semibold">Habit Execution Rate</span>
            <CheckSquare className="w-4 h-4 text-orange-400" />
          </div>
          <h4 className="text-2xl font-black font-mono text-slate-100">{avgHabitRate}%</h4>
          <p className="text-[10px] text-orange-400 font-medium">9 Core Goals Adherence</p>
        </div>

        <div className="p-4 bg-[#14100c] border border-orange-950/60 rounded-xl space-y-2">
          <div className="flex justify-between items-center text-xs">
            <span className="text-slate-400 font-semibold">Trading Plan Compliance</span>
            <TrendingUp className="w-4 h-4 text-emerald-400" />
          </div>
          <h4 className="text-2xl font-black font-mono text-slate-100">{planComplianceRate}%</h4>
          <p className="text-[10px] text-emerald-400 font-medium">Zero Revenge Trades Rule</p>
        </div>
      </div>

      {/* Weekly Breakdown Table */}
      <div className="space-y-3">
        <h4 className="text-xs font-bold text-slate-200 uppercase tracking-wider">7-Day Logged Activity Audit</h4>
        <div className="space-y-2">
          {last7DaysLogs.map((log) => {
            const liters = ((log.waterIntakeMl || 0) / 1000).toFixed(2);
            const habitsCount = (log.habitsCompleted || []).length;

            return (
              <div key={log.date} className="p-3 bg-[#14100c] border border-orange-950/40 rounded-xl flex justify-between items-center text-xs font-mono">
                <span className="text-slate-300 font-bold">{log.date}</span>
                <div className="flex items-center gap-4 text-slate-400">
                  <span>💧 <strong className="text-cyan-400">{liters} L</strong></span>
                  <span>🎯 <strong className="text-orange-400">{habitsCount}/9 Habits</strong></span>
```

```
                <span>💤 <strong className="text-slate-200">{log.sleepDuration || 8}h Sleep</strong></span>
              </div>
            </div>
          );
        })}
      </div>
    </div>
  </div>
 );
}

// ==========================================
// 10. MONEY MANAGER
// ==========================================
function MoneyModule({ transactions, addTransaction, currency, selectedDate, setData }) {
 const [type, setType] = useState('Expense');
 const [amount, setAmount] = useState('');
 const [category, setCategory] = useState('Food / Groceries');
 const [description, setDescription] = useState('');

 const handleAddTransaction = (e) => {
  e.preventDefault();
  if (!amount) return;

  const newTx = {
   id: 'tx_' + Date.now(),
   date: selectedDate,
   type,
   amount: Number(amount),
   category,
   description
  };

  addTransaction(newTx);
  setAmount(''); setDescription('');
 };

 return (
  <div className="grid grid-cols-1 lg:grid-cols-3 gap-6">
   <form onSubmit={handleAddTransaction} className="p-6 bg-[#0f141a] rounded-xl border border-slate-800/80 space-y-4">
    <h3 className="text-sm font-bold text-slate-100 flex items-center gap-2">
     <Wallet className="w-4 h-4 text-emerald-400" /> Record Financial Entry
```

```
        </h3>

        <div>
          <label className="text-xs text-slate-400 block mb-1">Type</label>
          <select value={type} onChange={(e) => setType(e.target.value)} className="w-full
bg-[#141a22] border border-slate-800 rounded p-2 text-xs text-slate-200">
            <option>Expense</option>
            <option>Saved</option>
            <option>Income</option>
          </select>
        </div>

        <div>
          <label className="text-xs text-slate-400 block mb-1">Amount ({currency})</label>
          <input type="number" value={amount} onChange={(e) => setAmount(e.target.value)}
className="w-full bg-[#141a22] border border-slate-800 rounded p-2 text-xs text-slate-200" />
        </div>

        <div>
          <label className="text-xs text-slate-400 block mb-1">Category</label>
          <input type="text" value={category} onChange={(e) => setCategory(e.target.value)}
className="w-full bg-[#141a22] border border-slate-800 rounded p-2 text-xs text-slate-200" />
        </div>

        <div>
          <label className="text-xs text-slate-400 block mb-1">Description</label>
          <input type="text" value={description} onChange={(e) => setDescription(e.target.value)}
className="w-full bg-[#141a22] border border-slate-800 rounded p-2 text-xs text-slate-200" />
        </div>

        <button type="submit" className="w-full py-2.5 bg-emerald-500 hover:bg-emerald-400
text-slate-950 font-bold text-xs rounded-lg transition-colors">
          Record Financial Transaction
        </button>
      </form>

      <div className="lg:col-span-2 p-6 bg-[#0f141a] rounded-xl border border-slate-800/80
space-y-4">
        <h3 className="text-sm font-bold text-slate-100">Financial Log</h3>
        <div className="space-y-2">
          {transactions.map((tx) => (
            <div key={tx.id} className="p-3 bg-[#141a22] rounded-lg border border-slate-800 flex
justify-between items-center text-xs">
              <div>
```

```
              <p className="font-bold text-slate-200">{tx.description || tx.category}</p>
              <p className="text-[10px] text-slate-500">{tx.type} • {tx.date}</p>
            </div>
            <span className={`font-mono font-bold ${tx.type === 'Saved' ? 'text-emerald-400' :
'text-slate-200'}`}>
              {currency}{tx.amount}
            </span>
          </div>
        ))}
      </div>
    </div>
  </div>
 );
}

// ==========================================
// 11. DAILY REFLECTION JOURNAL
// ==========================================
function JournalModule({ currentLog, updateDailyLog, selectedDate }) {
 const j = currentLog.journal || {};

 const handleSaveField = (field, value) => {
  updateDailyLog(selectedDate, {
   journal: { ...j, [field]: value }
  });
 };

 return (
  <div className="p-6 bg-[#0f141a] rounded-xl border border-slate-800/80 space-y-4">
   <h3 className="text-sm font-bold text-slate-100 flex items-center gap-2">
    <BookMarked className="w-4 h-4 text-emerald-400" /> Daily Reflection Journal
({selectedDate})
   </h3>

   <div className="space-y-4">
    <div>
     <label className="text-xs text-slate-400 block mb-1">What went well today?</label>
     <textarea value={j.wentWell || ''} onChange={(e) => handleSaveField('wentWell',
e.target.value)} className="w-full bg-[#141a22] border border-slate-800 rounded p-2.5 text-xs
text-slate-200 h-20" />
    </div>

    <div>
```

```
        <label className="text-xs text-slate-400 block mb-1">What went wrong or needed
improvement?</label>
        <textarea value={j.wentWrong || ''} onChange={(e) => handleSaveField('wentWrong',
e.target.value)} className="w-full bg-[#141a22] border border-slate-800 rounded p-2.5 text-xs
text-slate-200 h-20" />
      </div>

      <div>
        <label className="text-xs text-slate-400 block mb-1">What did I learn today?</label>
        <textarea value={j.learned || ''} onChange={(e) => handleSaveField('learned',
e.target.value)} className="w-full bg-[#141a22] border border-slate-800 rounded p-2.5 text-xs
text-slate-200 h-20" />
      </div>
    </div>
  </div>
 );
}

// ==========================================
// 13. AI DISCIPLINE COACH
// ==========================================
function AiCoachModule({ aiApiKey, setAiApiKey, callAiCoach, aiLoading, aiResponse,
aiPromptInput, setAiPromptInput }) {
 return (
  <div className="p-6 bg-[#0f141a] rounded-xl border border-slate-800/80 space-y-4">
    <h3 className="text-sm font-bold text-slate-100 flex items-center gap-2">
      <Bot className="w-5 h-5 text-emerald-400" /> AI Discipline Coach
    </h3>

    <div className="flex gap-2">
      <input
        type="text"
        value={aiPromptInput}
        onChange={(e) => setAiPromptInput(e.target.value)}
        placeholder="Ask AI Coach about trading discipline, nutrition, or sleep..."
        className="flex-1 bg-[#141a22] border border-slate-800 rounded p-2.5 text-xs
text-slate-200"
      />
      <button
        onClick={() => callAiCoach(aiPromptInput)}
        disabled={aiLoading}
        className="px-5 py-2.5 bg-emerald-500 hover:bg-emerald-400 text-slate-950
font-extrabold text-xs rounded transition-all"
      >
```

```
          {aiLoading ? 'Analyzing...' : 'Ask AI'}
        </button>
      </div>

      <div className="p-4 bg-[#141a22] rounded-xl border border-slate-800 text-xs font-mono
text-slate-200 whitespace-pre-line leading-relaxed">
        {aiResponse || "AI Coach Ready. Click 'Ask AI' to analyze your logged metrics."}
      </div>
    </div>
  );
}

// ========================================
// UTILITY & SUB-COMPONENTS
// ========================================
function AnalyticsKpiCard({ title, value, change, icon, accent }) {
  return (
    <div className="p-4 bg-[#0f141a] border border-slate-800/80 rounded-xl space-y-2">
      <div className="flex justify-between items-center text-xs">
        <span className="text-slate-400 font-semibold">{title}</span>
        {React.isValidElement(icon) ? icon : (icon && React.createElement(icon, { className: `w-4
h-4 ${accent}` }))}
      </div>
      <h4 className="text-2xl font-black font-mono text-slate-100">{value}</h4>
      <p className={`text-[10px] font-medium ${accent || 'text-emerald-400'}`}>{change}</p>
    </div>
  );
}

function ChartSummaryCard({ title, score, subtitle, status }) {
  return (
    <div className="p-4 bg-[#141a22] rounded-xl border border-slate-800 space-y-2">
      <div className="flex justify-between items-center text-xs">
        <span className="text-slate-300 font-bold">{title}</span>
        <span className="text-[10px] font-mono px-2 py-0.5 rounded bg-orange-950/80
text-orange-400 border border-orange-800 font-bold">{status}</span>
      </div>
      <h4 className="text-xl font-mono font-black text-orange-400">{score}</h4>
      <p className="text-[10px] text-slate-400">{subtitle}</p>
    </div>
  );
}
```

```
function LaunchpadModalBoard({ type, setType, onClose, currentLog, toggleHabit, addMeal, addWater }) {
  const [mealName, setMealName] = useState('');
  const [calories, setCalories] = useState('');
  const [waterMl, setWaterMl] = useState(3000);

  return (
    <div className="fixed inset-0 z-50 bg-black/85 backdrop-blur-md flex items-center justify-center p-4">
      <div className="bg-[#0f1015] border border-orange-500/40 w-full max-w-lg rounded-2xl p-6 space-y-4 shadow-2xl">
        <div className="flex justify-between items-center border-b border-orange-950/60 pb-3">
          <h3 className="text-sm font-black text-orange-400 uppercase tracking-wider flex items-center gap-2">
            <Zap className="w-4 h-4 text-orange-400 fill-orange-400" /> Daily Action Launchpad Board
          </h3>
          <button onClick={onClose} className="p-1 text-slate-400 hover:text-slate-100">
            <X className="w-5 h-5" />
          </button>
        </div>

        {type === 'goals' && (
          <div className="space-y-3 max-h-[60vh] overflow-y-auto pr-1">
            <p className="text-xs text-slate-400 font-mono">Tap any goal below to instantly complete or uncheck it for today:</p>
            {DEFAULT_HABITS.map(h => {
              const isChecked = (currentLog.habitsCompleted || []).includes(h.id);
              return (
                <div
                  key={h.id}
                  onClick={() => toggleHabit(h.id)}
                  className={`p-3.5 rounded-xl border cursor-pointer flex items-center justify-between transition-all ${
                    isChecked ? 'bg-orange-500/20 border-orange-500 text-orange-200 shadow-[0_0_12px_rgba(249,115,22,0.2)]' : 'bg-[#14100c] border-orange-950/60 text-slate-300 hover:border-orange-500/40'
                  }`}
                >
                  <div className="pr-3">
                    <h5 className="text-xs font-bold text-slate-100">{h.title}</h5>
                    <p className="text-[10px] text-orange-400/90 font-mono mt-0.5">{h.subtext}</p>
                  </div>
```

```
                <div className={`w-5 h-5 min-w-[20px] rounded flex items-center justify-center
border transition-all ${isChecked ? 'bg-orange-500 border-orange-400 text-slate-950' :
'border-slate-700'}`}>
                  {isChecked && <Check className="w-3.5 h-3.5 stroke-[3]" />}
                </div>
              </div>
            );
          })}
        </div>
      )}

      {type === 'meal' && (
        <form onSubmit={(e) => {
          e.preventDefault();
          if (mealName) {
            addMeal({ id: 'm_' + Date.now(), name: mealName, calories: Number(calories) || 500,
time: new Date().toLocaleTimeString([], { hour: '2-digit', minute: '2-digit' }) });
            onClose();
          }
        }} className="space-y-3">
          <div>
            <label className="text-xs text-slate-400 block mb-1">Meal Name</label>
            <input type="text" value={mealName} onChange={e => setMealName(e.target.value)}
placeholder="e.g. Steak, Eggs & Sweet Potato" className="w-full bg-[#14100c] border
border-orange-950/60 rounded p-2.5 text-xs text-slate-200" />
          </div>
          <div>
            <label className="text-xs text-slate-400 block mb-1">Calories (kcal)</label>
            <input type="number" value={calories} onChange={e => setCalories(e.target.value)}
placeholder="650" className="w-full bg-[#14100c] border border-orange-950/60 rounded p-2.5
text-xs text-slate-200" />
          </div>
          <button type="submit" className="w-full py-2.5 bg-orange-500 text-slate-950 text-xs
font-extrabold rounded-xl hover:bg-orange-400 transition-all">Save Meal</button>
        </form>
      )}

      {type === 'water' && (
        <div className="space-y-4 text-center">
          <h4 className="text-2xl font-mono font-extrabold text-cyan-400">
            {((currentLog.waterIntakeMl || 0) / 1000).toFixed(2)} L / 3.0 L
          </h4>
          <p className="text-xs text-slate-400 font-mono">Quick Hydration Log Inputs</p>
          <div className="grid grid-cols-2 gap-2">
```

```
            <button onClick={() => addWater(500)} className="p-3 bg-cyan-950/40 border
border-cyan-800 text-cyan-300 text-xs font-bold rounded-xl hover:bg-cyan-900/40
transition-all">+ 500 ml</button>
            <button onClick={() => addWater(1000)} className="p-3 bg-cyan-950/40 border
border-cyan-800 text-cyan-300 text-xs font-bold rounded-xl hover:bg-cyan-900/40
transition-all">+ 1.0 Liter</button>
            <button onClick={() => addWater(1500)} className="p-3 bg-cyan-950/40 border
border-cyan-800 text-cyan-300 text-xs font-bold rounded-xl hover:bg-cyan-900/40
transition-all">+ 1.5 Liters</button>
            <button onClick={() => addWater(3000)} className="p-3 bg-cyan-500 text-slate-950
text-xs font-extrabold rounded-xl hover:bg-cyan-400 transition-all">Full 3.0 L Target</button>
          </div>
        </div>
      )}
    </div>
  </div>
 );
}

// ==========================================
// 2. AI HEALTH & NUTRITION MODULE (CAL AI)
// ==========================================
function CalAiHealthModule({ data, currentLog, selectedDate, addMeal, addWater,
updateDailyLog }) {
 const [mealName, setMealName] = useState('');
 const [calories, setCalories] = useState('');
 const [protein, setProtein] = useState('');
 const [carbs, setCarbs] = useState('');
 const [fat, setFat] = useState('');

 const todaysMeals = data.meals.filter(m => m.date === selectedDate);
 const totalCalories = todaysMeals.reduce((acc, m) => acc + (Number(m.calories) || 0), 0);
 const totalProtein = todaysMeals.reduce((acc, m) => acc + (Number(m.protein) || 0), 0);
 const totalCarbs = todaysMeals.reduce((acc, m) => acc + (Number(m.carbs) || 0), 0);
 const totalFat = todaysMeals.reduce((acc, m) => acc + (Number(m.fat) || 0), 0);

 const waterMl = currentLog.waterIntakeMl || 0;
 const waterLiters = (waterMl / 1000).toFixed(2);
 const targetLiters = ((data.userProfile.waterTargetMl || 3000) / 1000).toFixed(1);

 const handleLogMeal = (e) => {
  e.preventDefault();
  if (!mealName) return;
```

```
    addMeal({
      id: 'm_' + Date.now(),
      date: selectedDate,
      name: mealName,
      calories: Number(calories) || 450,
      protein: Number(protein) || 30,
      carbs: Number(carbs) || 45,
      fat: Number(fat) || 15,
      time: new Date().toLocaleTimeString([], { hour: '2-digit', minute: '2-digit' })
    });

    setMealName(''); setCalories(''); setProtein(''); setCarbs(''); setFat('');
  };

  const nutritionRules = [
    { key: 'noSoda', label: 'No Soda' },
    { key: 'noAlcohol', label: 'No Alcohol' },
    { key: 'noSweetDrinks', label: 'No Sweet Drinks' },
    { key: 'noPackagedFoods', label: 'No Packaged Foods' },
    { key: 'noJunkFood', label: 'No Junk Foods' },
    { key: 'noSweets', label: 'No Sweets' },
    { key: 'noAddedSugar', label: 'No Sugar' },
  ];

  const currentNutrition = currentLog.nutrition || {};

  const toggleNutritionRule = (ruleKey) => {
    updateDailyLog(selectedDate, {
      nutrition: {
        ...currentNutrition,
        [ruleKey]: !currentNutrition[ruleKey]
      }
    });
  };

  return (
    <div className="space-y-6">
      <div className="p-5 bg-[#0f1015]/95 backdrop-blur-xl rounded-2xl border
border-orange-500/30 flex flex-wrap justify-between items-center gap-4 shadow-xl">
        <div>
          <h3 className="text-base font-black text-slate-100 flex items-center gap-2">
            <Utensils className="w-5 h-5 text-orange-400" /> AI Calorie & Health Tracker (CAL AI)
          </h3>
          <p className="text-xs text-orange-400/80 font-mono mt-0.5">
```

```
          Log meals, track daily macro targets, hydration, and strict dietary NOs.
        </p>
      </div>
      <div className="flex items-center gap-3">
        <div className="text-right font-mono">
          <span className="text-[10px] text-slate-400 block uppercase">Hydration Level</span>
          <span className="text-sm font-black text-cyan-400">{waterLiters} L / {targetLiters}
L</span>
        </div>
      </div>
    </div>

    {/* Macro Target Summary Cards */}
    <div className="grid grid-cols-2 sm:grid-cols-4 gap-4">
      <div className="p-4 bg-[#14100c] border border-orange-950/60 rounded-xl space-y-1">
        <span className="text-xs text-slate-400 font-semibold">Calories</span>
        <h4 className="text-xl font-black font-mono text-orange-400">{totalCalories} /
{data.userProfile.calorieTarget} kcal</h4>
        <p className="text-[10px] text-slate-500">Daily Energy Intake</p>
      </div>
      <div className="p-4 bg-[#14100c] border border-orange-950/60 rounded-xl space-y-1">
        <span className="text-xs text-slate-400 font-semibold font-mono">Protein</span>
        <h4 className="text-xl font-black font-mono text-emerald-400">{totalProtein}g /
{data.userProfile.proteinTarget}g</h4>
        <p className="text-[10px] text-slate-500">Muscle Repair</p>
      </div>
      <div className="p-4 bg-[#14100c] border border-orange-950/60 rounded-xl space-y-1">
        <span className="text-xs text-slate-400 font-semibold font-mono">Carbs</span>
        <h4 className="text-xl font-black font-mono text-amber-400">{totalCarbs}g /
{data.userProfile.carbsTarget}g</h4>
        <p className="text-[10px] text-slate-500">Glycogen Fuel</p>
      </div>
      <div className="p-4 bg-[#14100c] border border-orange-950/60 rounded-xl space-y-1">
        <span className="text-xs text-slate-400 font-semibold font-mono">Healthy Fats</span>
        <h4 className="text-xl font-black font-mono text-purple-400">{totalFat}g /
{data.userProfile.fatTarget}g</h4>
        <p className="text-[10px] text-slate-500">Hormonal Support</p>
      </div>
    </div>

    <div className="grid grid-cols-1 lg:grid-cols-3 gap-6">
      {/* Meal Logging Form */}
      <form onSubmit={handleLogMeal} className="p-6 bg-[#0f1015] rounded-xl border
border-orange-950/60 space-y-4 shadow-lg">
```

```
        <h4 className="text-xs font-bold text-orange-400 uppercase tracking-wider flex
items-center gap-2">
          <Plus className="w-4 h-4 text-orange-400" /> Log Meal Entry
        </h4>

        <div>
          <label className="text-xs text-slate-400 block mb-1">Meal Name / Description</label>
          <input
            type="text"
            value={mealName}
            onChange={e => setMealName(e.target.value)}
            placeholder="e.g. Steak, 3 Eggs, Rice & Avocado"
            className="w-full bg-[#14100c] border border-orange-950/60 rounded p-2.5 text-xs
text-slate-200"
          />
        </div>

        <div className="grid grid-cols-2 gap-2">
          <div>
            <label className="text-xs text-slate-400 block mb-1">Calories (kcal)</label>
            <input type="number" value={calories} onChange={e => setCalories(e.target.value)}
placeholder="650" className="w-full bg-[#14100c] border border-orange-950/60 rounded p-2
text-xs text-slate-200" />
          </div>
          <div>
            <label className="text-xs text-slate-400 block mb-1">Protein (g)</label>
            <input type="number" value={protein} onChange={e => setProtein(e.target.value)}
placeholder="45" className="w-full bg-[#14100c] border border-orange-950/60 rounded p-2
text-xs text-slate-200" />
          </div>
        </div>

        <div className="grid grid-cols-2 gap-2">
          <div>
            <label className="text-xs text-slate-400 block mb-1">Carbs (g)</label>
            <input type="number" value={carbs} onChange={e => setCarbs(e.target.value)}
placeholder="50" className="w-full bg-[#14100c] border border-orange-950/60 rounded p-2
text-xs text-slate-200" />
          </div>
          <div>
            <label className="text-xs text-slate-400 block mb-1">Fats (g)</label>
            <input type="number" value={fat} onChange={e => setFat(e.target.value)}
placeholder="18" className="w-full bg-[#14100c] border border-orange-950/60 rounded p-2
text-xs text-slate-200" />
```

```
          </div>
        </div>

        <button type="submit" className="w-full py-2.5 bg-orange-500 hover:bg-orange-400
text-slate-950 font-extrabold text-xs rounded-xl shadow-lg shadow-orange-950/80
transition-all">
          Save Meal & Update Macros
        </button>
      </form>

      {/* Meal History & Dietary NO Toggles */}
      <div className="lg:col-span-2 space-y-6">
        {/* Fast Hydration Quick Buttons */}
        <div className="p-5 bg-[#0f1015] rounded-xl border border-cyan-950/60 space-y-3">
          <div className="flex justify-between items-center">
            <h4 className="text-xs font-bold text-cyan-400 uppercase tracking-wider flex
items-center gap-2">
              <Droplets className="w-4 h-4 text-cyan-400" /> Daily Water Logger (Target 3.0 L)
            </h4>
            <span className="text-xs font-mono font-bold text-cyan-300">{waterLiters} L / 3.0
L</span>
          </div>
          <div className="grid grid-cols-2 sm:grid-cols-4 gap-2">
            <button onClick={() => addWater(250)} className="py-2 px-3 bg-cyan-950/40 border
border-cyan-800/80 text-cyan-300 text-xs font-bold rounded-xl hover:bg-cyan-900/50
transition-all">+250 ml</button>
            <button onClick={() => addWater(500)} className="py-2 px-3 bg-cyan-950/40 border
border-cyan-800/80 text-cyan-300 text-xs font-bold rounded-xl hover:bg-cyan-900/50
transition-all">+500 ml</button>
            <button onClick={() => addWater(1000)} className="py-2 px-3 bg-cyan-950/40 border
border-cyan-800/80 text-cyan-300 text-xs font-bold rounded-xl hover:bg-cyan-900/50
transition-all">+1.0 Liter</button>
            <button onClick={() => addWater(3000)} className="py-2 px-3 bg-cyan-500
text-slate-950 text-xs font-extrabold rounded-xl hover:bg-cyan-400 transition-all">Full 3L
Goal</button>
          </div>
        </div>

        {/* Strict Dietary NO Checklist */}
        <div className="p-5 bg-[#0f1015] rounded-xl border border-orange-950/60 space-y-3">
          <h4 className="text-xs font-bold text-slate-200 uppercase tracking-wider">7 Dietary
Discipline NOs</h4>
          <div className="grid grid-cols-2 sm:grid-cols-4 gap-2">
            {nutritionRules.map((rule) => {
```

```
            const isPassed = currentNutrition[rule.key] ?? true;
            return (
              <button
                key={rule.key}
                onClick={() => toggleNutritionRule(rule.key)}
                className={`p-2.5 rounded-xl border text-left text-xs font-bold flex items-center
justify-between transition-all ${
                  isPassed
                    ? 'bg-emerald-500/15 border-emerald-500/40 text-emerald-300'
                    : 'bg-rose-500/15 border-rose-500/40 text-rose-300'
                }`}
              >
                <span>{rule.label}</span>
                <span className="font-mono text-[10px]">{isPassed ? 'OK' : 'FAIL'}</span>
              </button>
            );
          })}
        </div>
      </div>

      {/* Logged Meals List */}
      <div className="p-5 bg-[#0f1015] rounded-xl border border-orange-950/60 space-y-3">
        <h4 className="text-xs font-bold text-slate-200 uppercase tracking-wider">Logged
Meals ({selectedDate})</h4>
        <div className="space-y-2">
          {todaysMeals.length === 0 ? (
            <p className="text-xs text-slate-500 font-mono">No meals recorded today. Add your
first meal using the logger form.</p>
          ) : (
            todaysMeals.map((m) => (
              <div key={m.id} className="p-3 bg-[#14100c] border border-orange-950/40
rounded-xl flex justify-between items-center text-xs">
                <div>
                  <h5 className="font-bold text-slate-100">{m.name}</h5>
                  <p className="text-[10px] text-slate-400 font-mono">{m.time} • P: {m.protein}g |
C: {m.carbs}g | F: {m.fat}g</p>
                </div>
                <span className="font-mono font-bold text-orange-400 bg-orange-950/60 px-2.5
py-1 rounded border border-orange-800">
                  {m.calories} kcal
                </span>
              </div>
            ))
          )}
```

```
          </div>
        </div>
      </div>
    </div>
  </div>
  );
}

// ========================================
// 3. LEARNING HUB (MEDIA) MODULE
// ========================================
function LearningHubModule({ learningHub, addLearningResource, setData }) {
  const [title, setTitle] = useState('');
  const [category, setCategory] = useState('Trading Skill');
  const [url, setUrl] = useState('');
  const [notes, setNotes] = useState('');

  const handleAddMedia = (e) => {
    e.preventDefault();
    if (!title) return;

    addLearningResource({
      id: 'l_' + Date.now(),
      title,
      category,
      url: url || '#',
      notes,
      completed: false,
      dateAdded: getTodayStr()
    });

    setTitle(''); setUrl(''); setNotes('');
  };

  const toggleMediaComplete = (id) => {
    setData(prev => ({
      ...prev,
      learningHub: prev.learningHub.map(item => item.id === id ? { ...item, completed:
!item.completed } : item)
    }));
  };

  const DEFAULT_RECOMMENDED_MEDIA = [
```

```
    { id: 'rec_1', title: 'Trading in the Zone - Mark Douglas Audio', category: 'Trading Skill', url:
'https://youtube.com', notes: 'Mastering psychological probabilities.' },
    { id: 'rec_2', title: 'Mental Growth & Emotional Mastery Podcast', category: 'Psychological', url:
'https://youtube.com', notes: 'Overcoming bias and impulsive behavior.' },
    { id: 'rec_3', title: 'Spiritual Discipline & Focus Routine', category: 'Spiritual', url:
'https://youtube.com', notes: 'Morning clarity and internal resilience.' },
  ];

  const allMedia = [...(learningHub || []), ...DEFAULT_RECOMMENDED_MEDIA];

  return (
    <div className="space-y-6">
      <div className="p-5 bg-[#0f1015]/95 backdrop-blur-xl rounded-2xl border
border-orange-500/30 flex justify-between items-center shadow-xl">
        <div>
          <h3 className="text-base font-black text-slate-100 flex items-center gap-2">
            <Video className="w-5 h-5 text-orange-400" /> Learning Media Hub (Podcasts &
Courses)
          </h3>
          <p className="text-xs text-orange-400/80 font-mono mt-0.5">
            Goal 06: Listen to Top Podcasts (Trading • Skill • Spiritual • Psychological • Mental
Growth)
          </p>
        </div>
      </div>

      <div className="grid grid-cols-1 lg:grid-cols-3 gap-6">
        {/* Media Add Form */}
        <form onSubmit={handleAddMedia} className="p-6 bg-[#0f1015] rounded-xl border
border-orange-950/60 space-y-4 shadow-lg">
          <h4 className="text-xs font-bold text-orange-400 uppercase tracking-wider flex
items-center gap-2">
            <Plus className="w-4 h-4 text-orange-400" /> Save Media / Podcast
          </h4>

          <div>
            <label className="text-xs text-slate-400 block mb-1">Title</label>
            <input type="text" value={title} onChange={e => setTitle(e.target.value)}
placeholder="e.g. Master Trader Psychology Ep. 12" className="w-full bg-[#14100c] border
border-orange-950/60 rounded p-2.5 text-xs text-slate-200" />
          </div>

          <div>
            <label className="text-xs text-slate-400 block mb-1">Category</label>
```

```
          <select value={category} onChange={e => setCategory(e.target.value)}
className="w-full bg-[#14100c] border border-orange-950/60 rounded p-2.5 text-xs
text-slate-200">
            <option>Trading Skill</option>
            <option>Spiritual</option>
            <option>Psychological</option>
            <option>Mental Growth</option>
          </select>
        </div>

        <div>
          <label className="text-xs text-slate-400 block mb-1">Media URL (YouTube / Podcast
Link)</label>
          <input type="text" value={url} onChange={e => setUrl(e.target.value)}
placeholder="https://..." className="w-full bg-[#14100c] border border-orange-950/60 rounded
p-2.5 text-xs text-slate-200" />
        </div>

        <div>
          <label className="text-xs text-slate-400 block mb-1">Key Takeaways & Notes</label>
          <textarea value={notes} onChange={e => setNotes(e.target.value)}
placeholder="Lessons learned..." className="w-full bg-[#14100c] border border-orange-950/60
rounded p-2.5 text-xs text-slate-200 h-20" />
        </div>

        <button type="submit" className="w-full py-2.5 bg-orange-500 hover:bg-orange-400
text-slate-950 font-extrabold text-xs rounded-xl shadow-lg shadow-orange-950/80
transition-all">
          Save Media Resource
        </button>
      </form>

      {/* Media Resource Library */}
      <div className="lg:col-span-2 p-6 bg-[#0f1015] rounded-xl border border-orange-950/60
space-y-4">
        <h4 className="text-xs font-bold text-slate-200 uppercase tracking-wider">Podcasts &
Learning Library</h4>
        <div className="space-y-3">
          {allMedia.map((item) => (
            <div key={item.id} className="p-4 bg-[#14100c] border border-orange-950/40
rounded-xl space-y-2">
              <div className="flex justify-between items-start">
                <div>
```

```
                <span className="text-[10px] font-mono font-bold px-2 py-0.5 rounded
bg-orange-950/80 text-orange-400 border border-orange-800">{item.category}</span>
                <h5 className="font-bold text-slate-100 text-sm mt-1">{item.title}</h5>
              </div>
              <button
                onClick={() => toggleMediaComplete(item.id)}
                className={`px-3 py-1 rounded-lg text-xs font-bold border transition-all ${
                  item.completed ? 'bg-emerald-500/20 border-emerald-500 text-emerald-300' :
'bg-orange-950/40 border-orange-800 text-orange-400'
                }`}
              >
                {item.completed ? 'COMPLETED' : 'MARK DONE'}
              </button>
            </div>
            {item.notes && <p className="text-xs text-slate-400">{item.notes}</p>}
            {item.url && item.url !== '#' && (
              <a href={item.url} target="_blank" rel="noreferrer" className="inline-flex
items-center gap-1 text-[11px] text-orange-400 hover:underline">
                <Play className="w-3 h-3 fill-orange-400" /> Watch / Listen Resource
              </a>
            )}
          </div>
        ))}
      </div>
    </div>
  </div>
 );
}

// ==========================================
// 5. FITNESS & GYM MODULE
// ==========================================
function FitnessModule({ workouts, addWorkout, selectedDate }) {
 const [workoutType, setWorkoutType] = useState('Gym / Push Workout');
 const [durationMinutes, setDurationMinutes] = useState('');
 const [notes, setNotes] = useState('');

 const handleAddWorkout = (e) => {
  e.preventDefault();
  if (!durationMinutes) return;

  addWorkout({
   id: 'w_' + Date.now(),
```

```
      date: selectedDate,
      type: workoutType,
      durationMinutes: Number(durationMinutes),
      notes: notes
    });

    setDurationMinutes('');
    setNotes('');
  };

  return (
    <div className="grid grid-cols-1 lg:grid-cols-3 gap-6">
      <form onSubmit={handleAddWorkout} className="p-6 bg-[#0f1015] rounded-xl border
border-orange-950/60 space-y-4">
        <h3 className="text-sm font-bold text-slate-100 flex items-center gap-2">
          <Dumbbell className="w-4 h-4 text-orange-400" /> Log Gym Workout
        </h3>

        <div>
          <label className="text-xs text-slate-400 block mb-1">Workout Type</label>
          <select value={workoutType} onChange={(e) => setWorkoutType(e.target.value)}
className="w-full bg-[#14100c] border border-orange-950/60 rounded p-2.5 text-xs
text-slate-200">
            <option>Gym / Heavy Strength Training</option>
            <option>Chest, Shoulders & Triceps (Push)</option>
            <option>Back & Biceps (Pull)</option>
            <option>Legs & Core</option>
            <option>Cardio & Endurance</option>
          </select>
        </div>

        <div>
          <label className="text-xs text-slate-400 block mb-1">Duration (Minutes)</label>
          <input type="number" value={durationMinutes} onChange={(e) =>
setDurationMinutes(e.target.value)} placeholder="60" className="w-full bg-[#14100c] border
border-orange-950/60 rounded p-2.5 text-xs text-slate-200" />
        </div>

        <div>
          <label className="text-xs text-slate-400 block mb-1">Workout Notes &
Performance</label>
          <textarea value={notes} onChange={(e) => setNotes(e.target.value)} placeholder="Sets,
reps, weights hit today..." className="w-full bg-[#14100c] border border-orange-950/60
rounded p-2.5 text-xs text-slate-200 h-20" />
```

```
        </div>

        <button type="submit" className="w-full py-2.5 bg-orange-500 hover:bg-orange-400
text-slate-950 font-extrabold text-xs rounded-xl shadow-lg shadow-orange-950/80
transition-all">
          Record Gym Session
        </button>
      </form>

      <div className="lg:col-span-2 p-6 bg-[#0f1015] rounded-xl border border-orange-950/60
space-y-4">
        <h3 className="text-sm font-bold text-slate-100">Workout Logs</h3>
        <div className="space-y-2">
          {workouts.length === 0 ? (
            <p className="text-xs text-slate-500 font-mono">No gym sessions logged yet. Record
one using the form on the left.</p>
          ) : (
            workouts.map((w) => (
              <div key={w.id} className="p-3 bg-[#14100c] border border-orange-950/40 rounded-xl
flex justify-between items-center text-xs">
                <div>
                  <h4 className="font-bold text-slate-100">{w.type}</h4>
                  <p className="text-[10px] text-slate-400 font-mono">{w.date} • {w.notes || 'No
notes'}</p>
                </div>
                <span className="font-mono font-bold text-orange-400 bg-orange-950/60 px-2.5
py-1 rounded border border-orange-800">
                  {w.durationMinutes} mins
                </span>
              </div>
            ))
          )}
        </div>
      </div>
    </div>
  );
}

// ==========================================
// 6. TRADING JOURNAL MODULE
// ==========================================
function TradingJournalModule({ trades, addTrade, selectedDate, currency }) {
  const [pair, setPair] = useState('EUR/USD');
  const [type, setType] = useState('BUY');
```

```
  const [resultR, setResultR] = useState('');
  const [pnl, setPnl] = useState('');
  const [followedPlan, setFollowedPlan] = useState(true);
  const [notes, setNotes] = useState('');

  const handleAddTrade = (e) => {
    e.preventDefault();
    if (!resultR) return;

    addTrade({
      id: 't_' + Date.now(),
      date: selectedDate,
      pair,
      type,
      resultR: Number(resultR),
      pnl: Number(pnl) || 0,
      followedPlan,
      notes
    });

    setResultR(''); setPnl(''); setNotes('');
  };

  return (
    <div className="grid grid-cols-1 lg:grid-cols-3 gap-6">
      <form onSubmit={handleAddTrade} className="p-6 bg-[#0f1015] rounded-xl border
border-orange-950/60 space-y-4">
        <h3 className="text-sm font-bold text-slate-100 flex items-center gap-2">
          <TrendingUp className="w-4 h-4 text-orange-400" /> Log Trade Entry
        </h3>

        <div>
          <label className="text-xs text-slate-400 block mb-1">Asset / Pair</label>
          <input type="text" value={pair} onChange={(e) => setPair(e.target.value)}
placeholder="EUR/USD" className="w-full bg-[#14100c] border border-orange-950/60
rounded p-2.5 text-xs text-slate-200" />
        </div>

        <div className="grid grid-cols-2 gap-2">
          <div>
            <label className="text-xs text-slate-400 block mb-1">Position Type</label>
            <select value={type} onChange={(e) => setType(e.target.value)} className="w-full
bg-[#14100c] border border-orange-950/60 rounded p-2.5 text-xs text-slate-200">
              <option>BUY</option>
```

```
            <option>SELL</option>
          </select>
        </div>
        <div>
          <label className="text-xs text-slate-400 block mb-1">Result (R-Multiple)</label>
          <input type="number" step="0.1" value={resultR} onChange={(e) =>
setResultR(e.target.value)} placeholder="+2.5" className="w-full bg-[#14100c] border
border-orange-950/60 rounded p-2.5 text-xs text-slate-200" />
        </div>
      </div>

      <div>
        <label className="text-xs text-slate-400 block mb-1">P&L Amount ({currency})</label>
        <input type="number" value={pnl} onChange={(e) => setPnl(e.target.value)}
placeholder="250" className="w-full bg-[#14100c] border border-orange-950/60 rounded p-2.5
text-xs text-slate-200" />
      </div>

      <div className="flex items-center gap-2 pt-1">
        <input type="checkbox" id="planCheck" checked={followedPlan} onChange={(e) =>
setFollowedPlan(e.target.checked)} className="rounded border-slate-700 accent-orange-500"
/>
        <label htmlFor="planCheck" className="text-xs text-slate-300 font-bold">Followed
Trading Plan 100% (No Revenge)</label>
      </div>

      <div>
        <label className="text-xs text-slate-400 block mb-1">Trade Notes & Rules Audit</label>
        <textarea value={notes} onChange={(e) => setNotes(e.target.value)}
placeholder="Confluence factors, risk management..." className="w-full bg-[#14100c] border
border-orange-950/60 rounded p-2.5 text-xs text-slate-200 h-20" />
      </div>

      <button type="submit" className="w-full py-2.5 bg-orange-500 hover:bg-orange-400
text-slate-950 font-extrabold text-xs rounded-xl shadow-lg shadow-orange-950/80
transition-all">
        Record Journal Entry
      </button>
    </form>

    <div className="lg:col-span-2 p-6 bg-[#0f1015] rounded-xl border border-orange-950/60
space-y-4">
      <h3 className="text-sm font-bold text-slate-100">Trade Execution Log</h3>
      <div className="space-y-2">
```

```
        {trades.length === 0 ? (
          <p className="text-xs text-slate-500 font-mono">No trades logged yet.</p>
        ) : (
          trades.map((t) => (
            <div key={t.id} className="p-3.5 bg-[#14100c] border border-orange-950/40
rounded-xl flex justify-between items-center text-xs">
              <div>
                <div className="flex items-center gap-2">
                  <span className={`font-mono font-bold px-2 py-0.5 rounded text-[10px] ${t.type
=== 'BUY' ? 'bg-emerald-500/20 text-emerald-400' : 'bg-rose-500/20
text-rose-400'}`}>{t.type}</span>
                  <h4 className="font-bold text-slate-100">{t.pair}</h4>
                  {t.followedPlan && <span className="text-[9px] font-mono bg-orange-950/80
text-orange-400 border border-orange-800 px-1.5 py-0.5 rounded">PLAN OK</span>}
                </div>
                <p className="text-[10px] text-slate-400 font-mono mt-1">{t.date} • {t.notes || 'No
execution notes'}</p>
              </div>
              <div className="text-right">
                <p className={`font-mono font-bold ${t.resultR >= 0 ? 'text-emerald-400' :
'text-rose-400'}`}>{t.resultR >= 0 ? `+${t.resultR}R` : `${t.resultR}R`}</p>
                <p className="text-[10px] text-slate-500 font-mono">{currency}{t.pnl}</p>
              </div>
            </div>
          ))
        )}
      </div>
    </div>
  </div>
 );
}

// ==========================================
// 7. TRADING ANALYTICS MODULE
// ==========================================
function TradingAnalyticsModule({ trades, currency }) {
 const totalTrades = trades.length;
 const wins = trades.filter(t => t.resultR > 0).length;
 const losses = trades.filter(t => t.resultR < 0).length;
 const winRate = totalTrades > 0 ? Math.round((wins / totalTrades) * 100) : 0;

 const totalR = trades.reduce((acc, t) => acc + (t.resultR || 0), 0).toFixed(1);
 const totalPnl = trades.reduce((acc, t) => acc + (t.pnl || 0), 0);
```

```
  const planAdherence = totalTrades > 0 ? Math.round((trades.filter(t => t.followedPlan).length / totalTrades) * 100) : 100;

  return (
    <div className="p-6 bg-[#0f1015] rounded-xl border border-orange-950/60 space-y-6">
      <div className="border-b border-orange-950/60 pb-3">
        <h3 className="text-base font-black text-slate-100 flex items-center gap-2">
          <BarChart3 className="w-5 h-5 text-orange-400" /> Trading Performance Analytics
        </h3>
        <p className="text-xs text-orange-400/80 font-mono mt-0.5">Statistical breakdown of edge execution and plan discipline.</p>
      </div>

      <div className="grid grid-cols-1 sm:grid-cols-4 gap-4">
        <div className="p-4 bg-[#14100c] border border-orange-950/40 rounded-xl space-y-1">
          <span className="text-xs text-slate-400">Total Win Rate</span>
          <h4 className="text-2xl font-black font-mono text-slate-100">{winRate}%</h4>
          <p className="text-[10px] text-orange-400">{wins} Wins / {losses} Losses</p>
        </div>

        <div className="p-4 bg-[#14100c] border border-orange-950/40 rounded-xl space-y-1">
          <span className="text-xs text-slate-400">Total Net R-Multiple</span>
          <h4 className={`text-2xl font-black font-mono ${Number(totalR) >= 0 ? 'text-emerald-400' : 'text-rose-400'}`}>{totalR}R</h4>
          <p className="text-[10px] text-slate-500">Cumulative Risk Units</p>
        </div>

        <div className="p-4 bg-[#14100c] border border-orange-950/40 rounded-xl space-y-1">
          <span className="text-xs text-slate-400">Net Profit & Loss</span>
          <h4 className="text-2xl font-black font-mono text-slate-100">{currency}{totalPnl}</h4>
          <p className="text-[10px] text-slate-500">Recorded P&L Total</p>
        </div>

        <div className="p-4 bg-[#14100c] border border-orange-950/40 rounded-xl space-y-1">
          <span className="text-xs text-slate-400">Plan Adherence</span>
          <h4 className="text-2xl font-black font-mono text-orange-400">{planAdherence}%</h4>
          <p className="text-[10px] text-orange-400 font-bold">Zero Revenge Rule</p>
        </div>
      </div>
    </div>
  );
}

// ==========================================
```

```
// 8. BOOK READING TRACKER
// ==========================================
function BookTrackerModule({ books, setData, selectedDate }) {
  const [title, setTitle] = useState('');
  const [author, setAuthor] = useState('');
  const [pagesRead, setPagesRead] = useState('');

  const handleAddBook = (e) => {
    e.preventDefault();
    if (!title) return;

    const newBook = {
      id: 'b_' + Date.now(),
      title,
      author,
      pagesRead: Number(pagesRead) || 10,
      date: selectedDate
    };

    setData(prev => ({
      ...prev,
      books: [newBook, ...(prev.books || [])]
    }));

    setTitle(''); setAuthor(''); setPagesRead('');
  };

  return (
    <div className="grid grid-cols-1 lg:grid-cols-3 gap-6">
      <form onSubmit={handleAddBook} className="p-6 bg-[#0f1015] rounded-xl border
border-orange-950/60 space-y-4">
        <h3 className="text-sm font-bold text-slate-100 flex items-center gap-2">
          <BookOpen className="w-4 h-4 text-orange-400" /> Log Daily Reading
        </h3>

        <div>
          <label className="text-xs text-slate-400 block mb-1">Book Title</label>
          <input type="text" value={title} onChange={(e) => setTitle(e.target.value)}
placeholder="Atomic Habits / Trading in the Zone" className="w-full bg-[#14100c] border
border-orange-950/60 rounded p-2.5 text-xs text-slate-200" />
        </div>

        <div>
          <label className="text-xs text-slate-400 block mb-1">Author</label>
```

```
        <input type="text" value={author} onChange={(e) => setAuthor(e.target.value)}
placeholder="Mark Douglas" className="w-full bg-[#14100c] border border-orange-950/60
rounded p-2.5 text-xs text-slate-200" />
      </div>

      <div>
        <label className="text-xs text-slate-400 block mb-1">Pages / Chapter Read</label>
        <input type="number" value={pagesRead} onChange={(e) =>
setPagesRead(e.target.value)} placeholder="10" className="w-full bg-[#14100c] border
border-orange-950/60 rounded p-2.5 text-xs text-slate-200" />
      </div>

      <button type="submit" className="w-full py-2.5 bg-orange-500 hover:bg-orange-400
text-slate-950 font-extrabold text-xs rounded-xl shadow-lg shadow-orange-950/80
transition-all">
        Save Reading Entry
      </button>
    </form>

    <div className="lg:col-span-2 p-6 bg-[#0f1015] rounded-xl border border-orange-950/60
space-y-4">
      <h3 className="text-sm font-bold text-slate-100">Reading Logs</h3>
      <div className="space-y-2">
        {books.length === 0 ? (
          <p className="text-xs text-slate-500 font-mono">No books logged yet. Aim for 10
pages or 1 chapter per day!</p>
        ) : (
          books.map((b) => (
            <div key={b.id} className="p-3 bg-[#14100c] border border-orange-950/40 rounded-xl
flex justify-between items-center text-xs">
              <div>
                <h4 className="font-bold text-slate-100">{b.title}</h4>
                <p className="text-[10px] text-slate-400 font-mono">{b.author || 'Unknown Author'}
• {b.date}</p>
              </div>
              <span className="font-mono font-bold text-orange-400 bg-orange-950/60 px-2.5
py-1 rounded border border-orange-800">
                {b.pagesRead} Pages
              </span>
            </div>
          ))
        )}
      </div>
    </div>
```

```
    </div>
  );
}

function HabitsModule({ currentLog, toggleHabit, shiftingTaskId }) {
  return (
    <div className="p-6 bg-[#0f1015] rounded-xl border border-orange-950/60 space-y-4">
      <h3 className="text-sm font-bold text-slate-100 flex items-center gap-2">
        <CheckSquare className="w-4 h-4 text-orange-400" /> Daily Core Habit Tracker
      </h3>
      <div className="space-y-2">
        {DEFAULT_HABITS.map((habit) => {
          const isDone = (currentLog.habitsCompleted || []).includes(habit.id);
          const isShifting = shiftingTaskId === habit.id;

          return (
            <div
              key={habit.id}
              onClick={() => toggleHabit(habit.id)}
              className={`p-3.5 rounded-xl border flex items-center justify-between cursor-pointer
transition-all ${
                isDone ? 'bg-orange-500/20 border-orange-500/60 text-orange-200' : 'bg-[#14100c]
border-orange-950/40 text-slate-300'
              } ${isShifting ? 'scale-105 transition-transform duration-300' : ''}`}
            >
              <div>
                <h4 className="text-xs font-bold">{habit.title}</h4>
                <p className="text-[10px] text-slate-400">{habit.subtext}</p>
              </div>
              <div className={`w-5 h-5 rounded flex items-center justify-center border ${isDone ?
'bg-orange-500 border-orange-400 text-slate-950' : 'border-slate-700'}`}>
                {isDone && <Check className="w-3.5 h-3.5 stroke-[3]" />}
              </div>
            </div>
          );
        })}
      </div>
    </div>
  );
}

function ChallengeGridModule({ data, selectedDate, setSelectedDate, setData }) {
  const activeDay = data.activeChallengeDay || 1;
  const completedList = data.completedDaysList || [];
```

```
  const handleCompleteActiveDay = () => {
    const nextDay = Math.min(90, activeDay + 1);
    const now = Date.now();

    setData(prev => ({
      ...prev,
      activeChallengeDay: nextDay,
      lastCompletedTimestamp: now,
      completedDaysList: Array.from(new Set([...(prev.completedDaysList || []), activeDay]))
    }));
  };

  return (
    <div className="p-6 bg-[#0f1015] rounded-xl border border-orange-950/60 space-y-6">
      <div className="flex justify-between items-center border-b border-orange-950/60 pb-3">
        <div>
          <h3 className="text-base font-black text-slate-100 flex items-center gap-2">
            <CalendarDays className="w-5 h-5 text-orange-400" /> 90-Day Challenge Master Grid
          </h3>
          <p className="text-xs text-orange-400/80 font-mono mt-0.5">Active Focus: Day
{activeDay} of 90</p>
        </div>
        <button
          onClick={handleCompleteActiveDay}
          className="px-4 py-2 bg-orange-500 hover:bg-orange-400 text-slate-950 font-extrabold
text-xs rounded-xl shadow-lg shadow-orange-950/80"
        >
          Complete Day {activeDay} & Shift →
        </button>
      </div>

      <div className="grid grid-cols-5 sm:grid-cols-10 gap-2">
        {Array.from({ length: 90 }, (_, i) => i + 1).map((dayNum) => {
          const isDone = completedList.includes(dayNum);
          const isActive = activeDay === dayNum;

          return (
            <div
              key={dayNum}
              className={`h-10 rounded-lg flex items-center justify-center text-xs font-mono
font-bold border transition-all ${
                isDone
                  ? 'bg-orange-500/20 border-orange-500 text-orange-300'
```

```
                : isActive
                ? 'bg-orange-500 border-orange-400 text-slate-950
shadow-[0_0_12px_rgba(249,115,22,0.5)]'
                : 'bg-[#14100c] border-orange-950/40 text-slate-600'
            }`}
           >
            {dayNum}
           </div>
         );
       })}
     </div>
   </div>
  );
}

// ==========================================
// 14. SETTINGS
// ==========================================
function SettingsModule({ data, setData, aiApiKey, setAiApiKey }) {
  const handleExportData = () => {
    const jsonStr = "data:text/json;charset=utf-8," + encodeURIComponent(JSON.stringify(data));
    const downloadAnchor = document.createElement('a');
    downloadAnchor.setAttribute("href", jsonStr);
    downloadAnchor.setAttribute("download",
`Wise_Discipline_OS_Backup_${getTodayStr()}.json`);
    document.body.appendChild(downloadAnchor);
    downloadAnchor.click();
    downloadAnchor.remove();
  };

  const handleRestartChallenge = () => {
    if (window.confirm("Are you sure you want to restart the 90-Day Challenge to Day 1 starting
today? This will clear past daily challenge completions.")) {
      const today = getTodayStr();
      setData(prev => ({
        ...prev,
        challengeStartDate: today,
        activeChallengeDay: 1,
        completedDaysList: [],
        dailyLogs: {
          [today]: {
            date: today,
            wakeTime: '07:00',
            bedTime: '23:00',
```

```
          sleepDuration: 8.0,
          waterIntakeMl: 0,
          showerCompleted: false,
          habitsCompleted: [],
          nutrition: { noSoda: true, noAlcohol: true, noSweetDrinks: true, noPackagedFoods: true,
noJunkFood: true, noSweets: true, noAddedSugar: true },
          journal: { achievements: '', wentWell: '', wentWrong: '', learned: '', improvementTomorrow:
'', mood: 4, disciplineScore: 0 }
        }
      }
    }));
  }
 };

 const handleFullReset = () => {
   if (window.confirm("WARNING: This will completely erase ALL logs, trade entries, workouts,
and reset the 90-Day Challenge to Day 1. Are you sure?")) {
     const today = getTodayStr();
     const resetState = {
       ...INITIAL_STATE,
       challengeStartDate: today,
       activeChallengeDay: 1,
       completedDaysList: [],
       dailyLogs: {
         [today]: {
           date: today,
           wakeTime: '',
           bedTime: '',
           sleepDuration: 0,
           waterIntakeMl: 0,
           showerCompleted: false,
           habitsCompleted: [],
           nutrition: { noSoda: true, noAlcohol: true, noSweetDrinks: true, noPackagedFoods: true,
noJunkFood: true, noSweets: true, noAddedSugar: true },
           journal: { achievements: '', wentWell: '', wentWrong: '', learned: '', improvementTomorrow:
'', mood: 3, disciplineScore: 0 }
         }
       },
       meals: [],
       trades: [],
       workouts: [],
       transactions: []
     };
     setData(resetState);
```

```
      localStorage.setItem(STORAGE_KEY, JSON.stringify(resetState));
    }
  };

  return (
    <div className="p-6 bg-[#0f141a] rounded-xl border border-slate-800/80 space-y-6">
      <div className="border-b border-slate-800 pb-3">
        <h3 className="text-sm font-bold text-slate-100 flex items-center gap-2">
          <Settings className="w-4 h-4 text-emerald-400" /> Application Settings & System
Control
        </h3>
        <p className="text-xs text-slate-400">Configure profile settings, Gemini API keys, and
challenge state.</p>
      </div>

      <div className="space-y-4 max-w-xl">
        <div>
          <label className="text-xs text-slate-400 block mb-1">Gemini AI API Key</label>
          <input
            type="password"
            value={aiApiKey}
            onChange={(e) => setAiApiKey(e.target.value)}
            placeholder="AI Key (Optional - Fallbacks to Offline Diagnostic Mode)"
            className="w-full bg-[#141a22] border border-slate-800 rounded p-2.5 text-xs
text-slate-200 font-mono"
          />
        </div>

        <div className="pt-4 border-t border-slate-800 space-y-3">
          <h4 className="text-xs font-bold text-slate-300 uppercase tracking-wider">Data &
Challenge Controls</h4>
          <div className="flex flex-wrap gap-3">
            <button
              onClick={handleExportData}
              className="px-4 py-2 bg-slate-800 hover:bg-slate-700 text-slate-200 text-xs font-bold
rounded-xl flex items-center gap-2 transition-all"
            >
              <Download className="w-4 h-4 text-emerald-400" /> Export Backup JSON
            </button>
            <button
              onClick={handleRestartChallenge}
              className="px-4 py-2 bg-amber-500/10 border border-amber-500/30 text-amber-300
hover:bg-amber-500/20 text-xs font-bold rounded-xl flex items-center gap-2 transition-all"
            >
```

```
            <RefreshCw className="w-4 h-4 text-amber-400" /> Restart Challenge from Day 1
          </button>
          <button
            onClick={handleFullReset}
            className="px-4 py-2 bg-rose-500/10 border border-rose-500/30 text-rose-300
hover:bg-rose-500/20 text-xs font-bold rounded-xl flex items-center gap-2 transition-all"
          >
            <Trash2 className="w-4 h-4 text-rose-400" /> Complete System Factory Reset
          </button>
        </div>
      </div>
    </div>
  </div>
 );
}
```